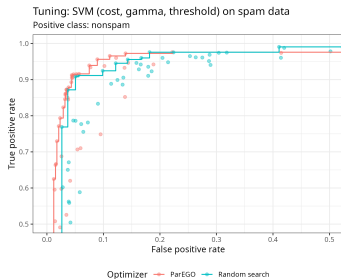
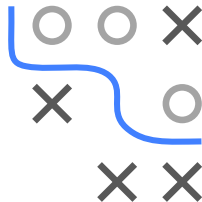


Multi-criteria Optimization Practical Applications



- ROC optimization
- Efficient models
- Fairness considerations

PRACTICAL APPLICATIONS IN MACHINE LEARNING

ROC Optimization: Balance *true positive* and *false positive* rates

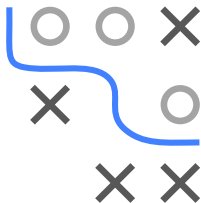
- Typically used in unbalanced classification with unspecified costs.
- Could also consider other ROC metrics, e.g., *positive predictive value* or *false discovery rate*.

Efficient Models: Balance *predictive performance* with *prediction time*, *energy consumption*, and/or *model size*.

- Time: production models need to predict fast.
- Size / Energy: model might be deployed on a mobile/edge device with power constraints.

Sparse Models: Balance *predictive performance* and *number of used features*—for cost efficiency or interpretability. **Fair Models:** Balance *predictive performance* and *fairness*.

- Model should not disadvantage certain subgroups (e.g. by gender).
- Multiple approaches exist to quantify fairness.



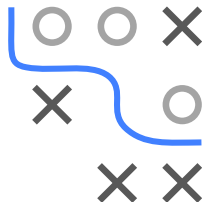
ROC OPTIMIZATION - SETUP

Again, we want to train a *spam detector* on the popular Spam dataset.

► [Click for source](#)

- Learner: SVM with RBF kernel
- Hyperparameters:
 - cost $[2^{-15}, 2^{15}]$
 - γ $[2^{-15}, 2^{15}]$
 - Threshold t $[0, 1]$
- Objective: *minimize* false positive rate (FPR) and *maximize* true positive rate (TPR), via 5-fold CV

- Optimizer: multi-criteria Bayesian optimization
 - ParEGO with $\rho = 0.05$, $s = 100000$
 - Acquisition: Confidence Bound ($\tau = 2.0$)
 - Budget: 100 evaluations
- Tuning is done on a training holdout, and the hyperparameter configs on the estimated Pareto front are validated on an external test set.



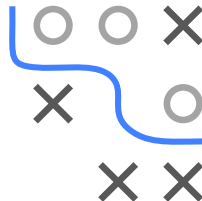
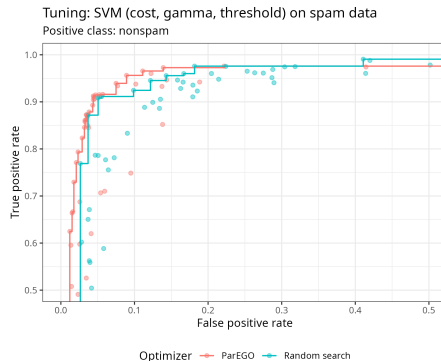
The threshold t could also be optimized post-hoc separately.

ROC OPTIMIZATION - RESULT I

- Compared to *random search*, many *ParEGO* evaluations lie on the Pareto front.
- The *ParEGO* Pareto front dominates most random-search solutions.
- Dominated hypervolume (w.r.t. reference point $(0, 1)$):

ParEGO: 0.965

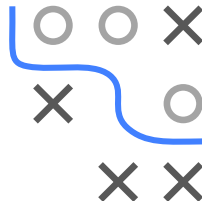
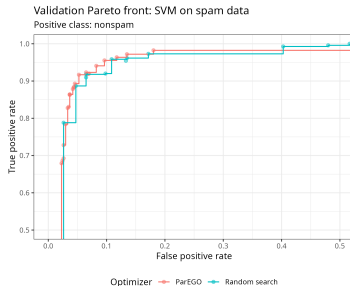
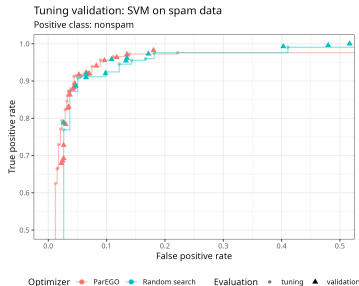
random: 0.959



ROC OPTIMIZATION - RESULT II

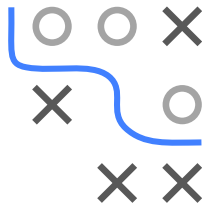
Validate configurations on the estimated Pareto front:

- The performance changes slightly on a new holdout set.
- TPR improved a bit, but FPR got slightly worse.
- Some configs become dominated once tested on the new holdout.
- Dominated hypervolume on the validation set:
ParEGO: 0.960
random: 0.961



EFFICIENT MODELS - OVERVIEW

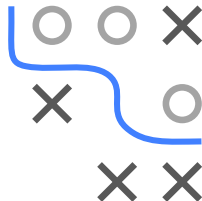
- “Efficiency” can refer to:
 - Memory consumption
 - Training or prediction time
 - Number of features used
 - Energy consumption
 - ...
- Some hyperparameters strongly impact efficiency:
 - Number of trees (RF, GBDT)
 - Number, size, type of layers (NN)
 - L1 regularization strength
- Others might have no direct influence on efficiency.
- Typical scenario: searching over multiple algorithms with different cost–performance trade-offs.



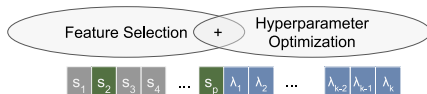
EFFICIENT MODELS - EXAMPLE: FEATURE SELECTION I

Goal: Identify an informative subset of features with minimal drop in performance vs. using all features.

$$\min_{\mathbf{x} \in \mathcal{S}, \mathbf{s} \in \{0,1\}^p} \left(\widehat{GE}(\mathcal{I}(\mathcal{D}, \mathbf{x}, \mathbf{s})), \frac{1}{p} \sum_{i=1}^p s_i \right).$$

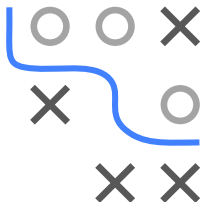


- Typically, feature selection and hyperparameter tuning happen in separate steps.
- Alternatively, search a good subset $\mathbf{s}$ and hyperparams $\mathbf{x}$ *simultaneously*.

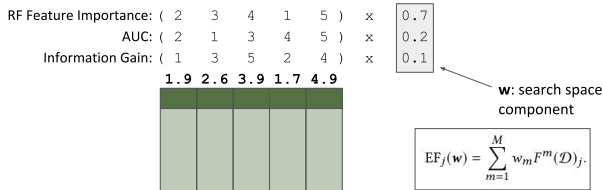


EFFICIENT MODELS - EXAMPLE: FEATURE SELECTION II

Idea: *Multi-Objective Hyperparameter Tuning and Feature Selection using Filter Ensembles* ▶ Binder et al. 2020 .



- Pre-calculate multiple *feature-filter* rank vectors.

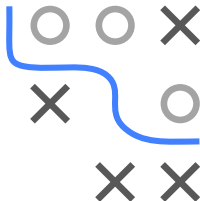
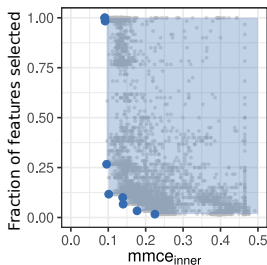


- New hyperparameter vector: $\mathbf{x} = (\tilde{\mathbf{x}}, w_1, \dots, w_p, \tau)$
 - $\tilde{\mathbf{x}}$: learner hyperparameters
 - $(w_1, \dots, w_p)$: weights for each filter ranking
 - $\tau \in [0, 1]$: fraction of features to keep

EFFICIENT MODELS - EXAMPLE: FEATURE SELECTION III

Joint FS + HP search on Sonar data ¹:

- Learner: SVM (RBF kernel).
- Hyperparams:
 - $\text{cost} \quad [2^{-10}, 2^{10}]$
 - $\gamma \quad [2^{-10}, 2^{10}]$
 - $(w_1, \dots, w_p) \quad [0, 1]^p$
 - $\tau \quad [0, 1]$
- Objective: minimize misclassification + fraction of selected features.
- Optimizer: ParEGO + random forest surrogate + LCB acquisition, 15 batch proposals, budget 2000 evals.



¹Only tuning error is shown

EFFICIENT MODELS - EXAMPLE: FLOPS

Goal: Optimize *accuracy* vs. *FLOPS* (floating-point ops)

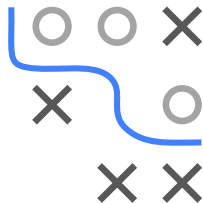
► Wang et al. 2019 .

Data: CIFAR-10 (image classification).

Learner: *DenseNet* ► Huang et al. 2017 :

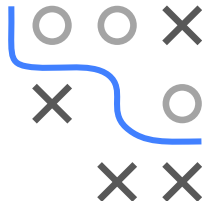
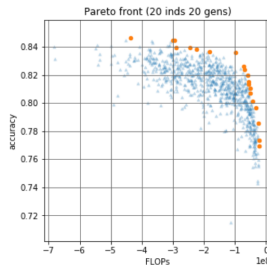
- 4 dense blocks, each with multiple convolution layers that feed into each other.
- Blocks connected by convolution + max-pooling layers.

Training: 300 epochs, batch size 128, initial LR 0.1.



EFFICIENT MODELS - EXAMPLE: FLOPS (CONT.)

- Objectives: *accuracy* vs. *FLOPS* per *observation*.
- Search space:
 - growth rate (k) [8, 32]
 - layers in 1st block [4, 6]
 - layers in 2nd block [4, 12]
 - layers in 3rd block [4, 24]
 - layers in 4th block [4, 16]
- Tuner: *Particle Swarm Optimization*, population 20, budget 400.

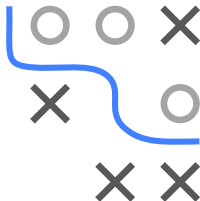
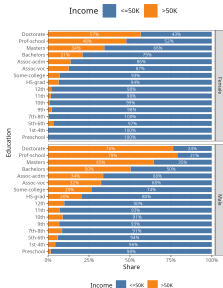
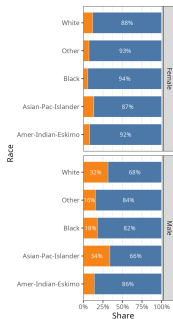
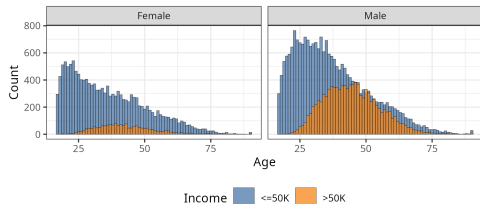


“Growth rate” = # of feature maps added each layer.

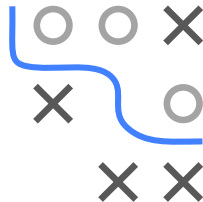
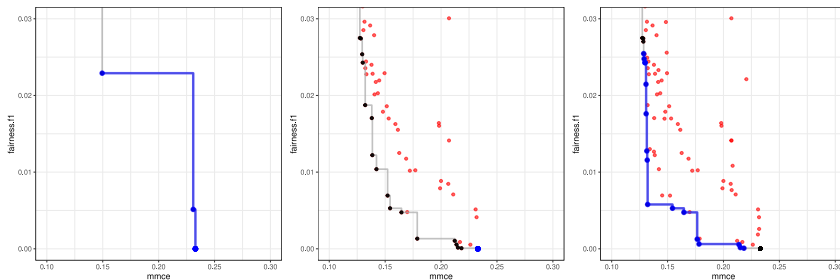
FAIR MODELS - THE ADULT DATASET

Dataset: Adult

- Source: US Census database, 1994.
▶ [Click for source](#)
- 48842 observations
- Target: binary, income above 50k
- 14 features: age, education, hours.per.week, marital.status, native.country, occupation, race, relationship, sex, ...



FAIR MODELS - RESULTS



- Optimizer: ParEGO + random forest surrogate, restricting $[w_i]$ to $[0.1, 0.9]$ if we only care about moderate fairness/performance ranges.
- In this example, hyperparams do indeed influence the fairness measure.
- But in practice, changing standard HPs often *won't* suffice to ensure fairness.